

解答冊子

- 解答冊子
 - 1章
 - 1.1.2 CPUとは
 - 1.1.3 メモリとは
 - 1.2.2 ビットとデータ表現
 - 1.2.3 2進数と10進数、16進数
 - 1.2.4 2進数と負の数（二の補数表現）
 - 3章
 - 3.2 くだいくらいの挨拶を
 - 3.4 おまけ問題
 - 2章
 - 2.2.1.0 LD ロード（load）
 - 2.2.1.1 ST ストア（store）
 - 2.2.1.2 LAD ロードアドレス（load address）
 - 2.2.2 算術・論理演算命令
 - 2.2.3.0 AND 論理積
 - 2.2.3.2 XOR 排他的論理和（exclusive OR）
 - 2.2.4 比較演算命令
 - 2.2.5.0 SLA 算術左シフト（shift left arithmetic）
 - 2.2.5.1 SRA 算術右シフト（shift right arithmetic）
 - 2.2.6 分岐命令
 - 2.2.7 スタック操作命令
 - 2.2.8 サブルーチン関係の命令
 - 2.3 アセンブラ命令

1章

1.1.2 CPUとは

問題

- (1) コンピュータの五大要素を全て述べよ。
- (2) CPUは、コンピュータの五大要素でいう 何 と 何 を担っているか。

解答

- (1) 制御装置、演算装置、記憶装置、入力装置、出力装置 （順不同）
- (2) 制御装置 と 演算装置

1.1.3 メモリとは

問題

- (1) 書き込みが自由にできるメモリをアルファベット三文字で何というか。
- (2) (1) に対して、書き込みが自由に**できない**メモリをアルファベット三文字で何というか。
- (3) メモリにおいて、データを格納する一つ一つの場所を指す番号を何というか。
- (4) 上の図について、A が格納されている (2) は、いくつか。

解答

- (1) RAM
- (2) ROM
- (3) 「アドレス」（あるいは「番地」でも可）
- (4) 3 番地

1.2.2 ビットとデータ表現

問題

- (1) 0 と 1 の組み合わせにより表現された数字列を、何というか。
- (2) (1) を構成するそれぞれの 0 や 1 を何と呼ぶか。
- (3) (1) を 8つ区切り にしたとき、それぞれをどういった単位で呼ぶか。
- (4) asciiコードにおいて、 3 は 何列目の何行目 か。
- (5) asciiコードにおいて、 . は 何列目の何行目 か。

解答

- (1) ビット列
- (2) ビット
- (3) バイト
- (4) 3列目 の 3行目
- (5) 2列目 の 14行目

1.2.3 2進数と10進数、16進数

問題

- (1) 173 を 8桁の2進数と、16進数に直しなさい
(2) 01001011 を 10進数に直しなさい

解答

- (1) 10101101、0xAD

173 は $128 + 32 + 8 + 4 + 1$ である。

よって、累乗に直すと $2^7 + 0 + 2^5 + 0 + 2^3 + 2^2 + 0 + 2^0$ だから、
10101101 となる。

また、16進数で 1010 は A、1101 は D であるから、173 は 0xAD となる。

- (2) 75

まず、二進数を 2の累乗 に直したい。

$01001011 = 0 + 2^6 + 0 + 0 + 2^3 + 0 + 2^1 + 2^0$ となるから、

$01001011 = 64 + 8 + 2 + 1$ であり、

75 が答えとなる。

1.2.4 2進数と負の数（二の補数表現）

問題

- (1) 二進数8桁で -20 を表現せよ。ヒント：通常の 20 は 00010100 である。
(2) 10000110 は、符号なし二進数でいくつか。また、符号付き二進数でいくつか。

解答

- (1) 11101100

20 (00010100) を、ビット反転すると 11101011 になる。

これに +1 すると 11101100 になる。

- (2) 符号なし：134、符号付き：-122

符号なし

$10000110 = 2^7 + 0 + 0 + 0 + 0 + 2^2 + 2^1 + 0$ より、

$10000110 = 128 + 4 + 2$ であり、計算して 134 となる。

符号あり

一番左の符号ビットが 1 なので負の数になる。符号なしと同様に変換できない。

正の数に変換する。ビット反転して 01111001 となり、+1 して 01111010 となる。

これを10進数に直すと、

$01111010 = 2^6 + 2^5 + 2^4 + 2^3 + 2^1$

$01111010 = 64 + 32 + 16 + 8 + 2$ より 122 となる。

負の数なので、マイナスをつけて -122 が答え。

おまけ

二の補数表現では、正の数 → 負の数 だけではなく、負の数 → 正の数 の変換も行うことが出来る。
これを利用すると、「2回同じ手順を繰り返すと、元に戻る」ことが分かる。

$20 (00010100) \rightarrow -20 (11101100) \rightarrow 20 (00010100)$ といったように。

この「2回行くと元に戻る」性質を **involution** と言ったりする。

名前を覚えなくても良いが、戻ってくる性質のことは知っていると面白いかもしれない。

3章

3.2 くどいくらいの挨拶を

問題

GR0 の初期値を 3 にして、5回繰り返してみよう。

解答

「繰り返すには、limit の値はどうなっていると良いか」を考える。

1行目を出力したときは、GR0 は $3 + 1 = 4$ になる。 よって、繰り返すには $4 < \text{limit}$ である。

2行目を出力したときは、GR0 は $4 + 1 = 5$ になる。 同様に、繰り返すには $5 < \text{limit}$ である。

3行目を出力したときは、GR0 は $5 + 1 = 6$ になる。 $6 < \text{limit}$ なら繰り返す。

4行目を出力したときは、GR0 は $6 + 1 = 7$ になる。 $7 < \text{limit}$ なら繰り返す。

5行目を出力したときは、GR0 は $7 + 1 = 8$ になる。 $8 < \text{limit}$ なら繰り返す。

よって、5回目のときに $8 < \text{limit}$ を満たさないなら、これ以上繰り返されない。

limit = 8 が適切と分かる。

```
MAIN      START
          LAD      GR0, init      ; GR0 をカウンタとして使う。初期値 3 を代入

FOR        OUT      string, len   ; 繰り返したい部分の先頭を FOR とラベル付け  <----┐
          ADDA     GR0, one       ; カウンタを 1 増やす                        │
          CPL      GR0, limit     ; カウンタ と 繰り返し回数 を比較          │
          JMI      FOR           ; カウンタ < 繰り返し回数 なら FOR に戻る  ----┘

          RET

string DC    'Hello, World!'
len    DC    13
limit  DC    8          ; 繰り返し回数。8に変更。
init   DC    3          ; カウンタの初期化に使う値。3に変更。
one     DC    1
          END
```

3.4 おまけ問題

問題

今のプログラムは、何行ぶん出力したのかが分かりにくい。

そこで、`1: Hello, World!` のように、先頭に何個目の Hello, World! なのか分かるようにしたい。

解答例

```
MAIN      START
          IN      limit, =1      ; 繰り返し回数を標準入力

          LD      GR0, limit     ; 移植部分
          SUBA    GR0, =#0030
          ST      GR0, limit

          LAD     GR0, 0         ; GR0 をカウンタとして使う。初期値0 を代入

FOR        ADDA   GR0, =1       ; カウンタを 1 増やす

; ここに、数値→数字の変換を記述
          LD      GR1, GR0       ; カウンタを直接弄らなくていいように、GR1 にコピー
          ADDA    GR1, =#0030    ; 数値 + 0x30 すれば 数字になる
; row に、変換した数字を格納
          ST      GR1, row
; 文字列を出力
          OUT     row, len       ; 先頭は row 。stringだと ': Hello, World!'から出力が始まる

          CPL     GR0, limit     ; カウンタ と 繰り返し回数 を比較
          JMI     FOR           ; カウンタ < 繰り返し回数 なら FOR に戻る

          RET

row        DS      1
string     DC      ': Hello, World!'
len        DC      16           ; row が 1文字、': ' が 2文字、全体で 1 + 2 + 13 = 16文字
limit      DS      1           ; 繰り返し回数
          END
```

2章

2.2.1.0 LD ロード (load)

問題

GR5 に値 120 を格納する。

解答例

```
MAIN      START
          LD      GR5, =120      ; せっかくなのでリテラルを使ってみる。サンプルのようにラベルを使っても良い
          RET
          END

-----
; ラベルを使う場合
MAIN      START
          LD      GR5, A
          RET
A          DC      120
          END
```

2.2.1.1 ST ストア (store)

問題

アドレス 0x0015 に ST 命令を使用して値 500 を格納する。

解答例

```
MAIN      START
          LD      GR0, =500      ; せっかくなのでリテラルを使ってみる。もちろんラベルを使っても良い
          ST      GR0, #0015     ; アドレスは 0x がついている16進数。10進数で 21 を指定しても良い
          RET
          END

-----
; ラベル、10進数を使う場合
MAIN      START
          LD      GR0, A
          ST      GR0, 21
          RET
A          DC      500
          END
```

2.2.1.2 LAD ロードアドレス (load address)

問題

ラベル `VALUE` を作成し、値 1 を格納する。

次に、その一行下に `DC 3` を行う。

最後に、指標レジスタを上手く使い `VALUE` のアドレスの1つ後を指定し、入っているデータを `GR0` に入れる。

解答例

指標レジスタに 1 が入ったレジスタを指定することで、実効アドレスを `VALUE + 1` にズラす。

方法は

1. `GR1` など、好きなレジスタ (`GR0` 以外) に 1 を格納する
2. `LD GR0, VALUE, レジスタ` とする

1 を格納するには、今回習った `LAD` を使ってほしい。

ラベルを追加して `DC` する、`LD` とリテラルを使う、なども可能だが、せっかくなので。

```
MAIN      START
          LAD    GR1, 1          ; GR1 を指標レジスタとする。1を入れる
          LD     GR0, VALUE, GR1 ; VALUE のアドレスの 1つ後 のアドレスに入っているデータを GR0
          RET
VALUE     DC     1
          DC     3
          END
```

2.2.2 算術・論理演算命令

問題（難しめ）

1. ADDA 命令では 0 となり、ADDL 命令ではオーバーフローする足し算を実装せよ
2. 引き算は、符号を変えた足し算として行われる。

$100 + (-2)$ ($0000000001100100 + 1111111111111110$) のように、

単に桁が溢れるだけの計算では、オーバーフローは起きない。

では、桁あふれ と オーバーフロー の違いは何か

解答例

1. 0 以外の値で、 a と $-a$ を足す。

ADDA (符号付きの足し算) では結果が 0 だから、 $1 + (-1)$ や $2 + (-2)$ のように、同じ大きさの異符号を考える ($a + (-a)$)。

このうち、 $0 + (-0)$ はオーバーフローしないから対象外となる。

```
MAIN      START
          LD      GR0, A
          LD      GR1, B
          ADDL    GR0, GR1
          RET
A         DC      1
B         DC      -1
          END
```

2. 桁あふれとオーバーフローの違いは、符号が関係する。

同じ符号で足し算を行った結果、違う符号になってしまった場合にオーバーフローとする。

本来、同じ符号を足すから $1 + 3 = 4$ や $-13 + (-100) = -113$ など、どう頑張っても同じ符号の値が得られるはずだ。

しかし、表現できる値の範囲を超え桁が溢れると、符号が変わってしまう。

符号なしの場合も、表示されていない9桁目に符号ビットが隠れている とすると、辻褃が合う。

この「表現できる値の範囲を超え桁が溢れる」をオーバーフローとする。

$50000 + 50000$ などを行い、得られる2進数の最上位ビットを確認してほしい。

2.2.3.0 AND 論理積

問題

任意の値を、AND を使って 0 に変えるにはどうしたらよいか

解答例

0 と論理積を取れば良い。

1 AND 0 は 0 になるし、0 AND 0 も 0 になる。

よって、0 と論理積を取ることで、どんな値でも 0 に変えることが出来る。

```
; -100 (1111111110011100) と 0 (0000000000000000) の論理積をとる
MAIN  START
      LD      GR0, A
      LD      GR1, B
      AND     GR0, GR1 ; GR0 が 0 になるはず
      RET
A      DC     -100      ; 好きな値に変える
B      DC     0         ; 0 を用意する
      END
```

2.2.3.2 XOR 排他的論理和 (exclusive OR)

問題

1. 任意の値について、全てのビットを反転させるにはどうしたらよいか
2. これを使って、「二の補数表現」を生み出すプログラムを作ってみよう

解答例

1. #FFFF と排他的論理和を取る。

#FFFF はすべてのビットが 1 である。

0 XOR 1 は 1 になり、1 XOR 1 は 0 になるから、16ビットでは #FFFF と排他的論理和を取ればいい。

2. 二の補数表現は、「ビットを反転したものに 1 を足す」ことで得られる。

よって、#FFFF と排他的論理和を取り、ADDA や LAD で 1 を足す。

```
; 100 (0000000001100100) を -100 (1111111110011100) に変換する
MAIN  START
      LD      GR0, A
      LD      GR1, B
      XOR     GR0, GR1
      ADDA    GR0, =1 ; 1 を足す。リテラルを使ってみた
      RET
A      DC     100      ; 好きな値に変える
B      DC     #FFFF    ; FFFF を用意する
      END
```

2.2.4 比較演算命令

問題（難しめ）

CPA と CPL では、FR の値が異なる結果になることがある。

では、CPA では FR = 000 となり、CPL では FR = 010（SFが1）となる計算を考えてみよう。

解答例

二つの数字を a と b とする。

ポイントは、「符号付きだと a の方が大きい」であり、符号なしだと「符号なしだと b の方が大きい」だ。

負の数は、符号なしだと「符号ビットが数値 $2^7 = 128$ になる」から、非常に大きな値になる。

よって、 a : 正の数 と b : 負の数 を比較すると良い！

```
MAIN      START
          LD      GR0, A
          LD      GR1, B
          CPL     GR0, GR1
          RET
A         DC      1
B         DC      -1
          END
```

2.2.5.0 SLA 算術左シフト (shift left arithmetic)

問題

算術左シフトを使って、値 57 を 10倍 してみよう

解答例

ヒントにある通り、8倍 と 2倍 を考える。

それぞれを中間値として、別のレジスタに保持しておくことで、後で足し合わせることが出来る。

```
MAIN  START
      LD    GR0, A    ; 2倍の保持用
      SLA   GR0, 1    ; 2倍する

      LD    GR1, A    ; 8倍の保持用。既に使った GR0 以外を指定
      SLA   GR1, 3    ; 8倍する

      ADDA  GR0, GR1  ; 2倍 + 8倍 で 10倍になる
      RET

A      DC    57
      END
```

2.2.5.1 SRA 算術右シフト (shift right arithmetic)

問題

左シフトでは掛け算が行えた。右シフトでは割り算になる。

上の例では、1桁右シフトで $\div 2$ 、2桁右シフトで $\div 4$ の操作に値する。結果を確認してほしい。

しかし、割り算は「割り切れないとき」が存在する。では、**あまり** はどこに行くのだろうか？

解答例

あふれたビット、つまり OVの値 があまりに相当する。

もう少し踏み込んで言うと、「あふれたビットを、逆順に並べた値」が余りとなる。

例えば 18 (10010) を 2桁右シフトして $\div 4$ をする。

まず、1桁右シフトすると、9 (1001) で OV は 0 になる。

次に、もう1桁右シフトすると、4 (100) で OV は 1 になる。

あふれたビットを逆順に並べると 10 となり、余り 2 と分かる。

17 $\div$ 4 の場合は 1, 0 の順であふれるから、01 で 余り 1 となる。

190 $\div$ 8 の場合は 0, 1, 1 の順であふれるから、110 で 余り 6 となる。

2.2.6 分岐命令

問題

1 から 100 までの和の合計を求め、`sum` ラベルに格納する。

つまり、 $1 + 2 + 3 + 4 + \dots + 99 + 100$ の答えを求め、`sum` に保存する

解答例

```
MAIN      START
          LAD      GR0, 0      ; GR0 を 0 で初期化する
          LAD      GR1, 0      ; GR1 を計算結果とする。0 で初期化する
FOR        ADDA     GR0, =1     ; GR0 を 1 増やす (1, 2, 3, ... と変えていく)
          ADDA     GR1, GR0     ; GR1 = GR1 + GR0
          CPL      GR0, count   ; GR0 と 100 を比較
          JMI      FOR          ; GR0 < 100 なら繰り返す
          ST       GR1, sum      ; sum に計算結果を保存
          RET
count     DC       100
sum       DS       1
          END
```

2.2.7 スタック操作命令

問題

スタックに 1, 2, 3, 4 の順で値を積んで、以下のようにレジスタに戻せ。

GR0 → 4, GR1 → 1, GR2 → 3, GR3 → 2

解答例

スタックから POP すると、4, 3, 2, 1 の順で値が出てくる。

よって、GR0, GR2, GR3, GR1 の順で受け取れば良い。

```
MAIN      START
          PUSH    1
          PUSH    2
          PUSH    3
          PUSH    4
          POP     GR0
          POP     GR2
          POP     GR3
          POP     GR1
          RET
          END
```

2.2.8 サブルーチン関係の命令

問題（お遊びコーナー）

永遠に終わらないプログラムを作ってみよう！

方法1：分岐命令で自分自身、あるいはそれ以前のメモリ番地に飛ぶ

方法2：PUSH と RET を使って前に戻る

解答例

方法1

```
MAIN    START
LOOP    JUMP    LOOP    ; 自分自身に飛ぶ。つまり、ここをもう一度実行
        RET
        END
```

固まったように見えるが、実行ログをスクロールすると勝手に一番下まで戻される。動いてはいるようだ。

```
LOOP    NOP
        JUMP    LOOP
```

とすると分かりやすいかも

方法2

```
MAIN    START
LOOP    PUSH    LOOP    ; 自分自身のアドレスをスタックに積む
        RET      ; スタックの値を PC に設定する。つまり LOOP に戻る
        END
```

2.3 アセンブラ命令

問題

さて、ここまで説明した内容で「おまじない」がすべて説明された。
ということで、次のプログラムについて、上から下まで全て解釈してみよう。

```
MAIN      START  PRG1
           LD      GR2, =30
PRG1       LD      GR0, A
           LAD     GR1, 50
           SUBL    GR1, GR0
           ST      GR1, B
           RET
A          DC      100
B          DS      1
           END
PRG2       LD      GR0, C
           RET
C          DC      130
```

解答例

まず、プログラムの実行開始位置を `PRG1` に設定する

`GR2` にリテラルを使って 30 を格納する（**実行されない**）

`PRG1` の宣言。 `START` の実行開始位置指定により、ここから実行される。

`GR0` に `A` ラベルの中身（100）を格納する。この処理をする命令のアドレスに `PRG1` の名前が紐づく。

`GR1` に 50 を格納する。

論理減算で `GR1` から `GR0` を引く。論理なので `-GR0` (-100) は 65436 として解釈される。

結果は $50 + 65436 = 65486$

`B` ラベルのついたメモリ番地に `GR1` の中身（65486）を格納する

スタックに何もないので、`RET` で終了する

`A` ラベルの付いた領域に 100 を入れる

`B` ラベルから 1語ぶん 領域を確保する

プログラムの終了。以降の3行は解釈されず、機械語に変換されない。実行もされない。

- `PRG2` の宣言。 `GR0` に `C` ラベルの中身（130）を格納する。
- `RET` 命令。
- `C` ラベルの付いた領域に 130 を入れる。

ちなみに、これを機械語に直すと以下ようになる（16進数）

1020 000C 1000 000A 1210 0032 2710 1110 000B 8100 0064 FFFF 001E